

Virtual Reality Benchmark for Edge Caching Systems

Nader Alfares*

Pennsylvania State University

George Kesidis†

Pennsylvania State University

ABSTRACT

We introduce a Unity based benchmark for evaluating Virtual Reality (VR) delivery systems using edge-cloud caching. As VR applications and systems progress, the need to meet strict latency and Quality of Experience (QoE) requirements is increasingly evident. In the context of VR, traditional cloud architectures (e.g., remote AWS S3 for content delivery) often struggle to meet these demands, especially for users of the same application in different locations. With edge computing, resources are brought closer to users in efforts to reduce latency and improve QoEs. However, VR's dynamic nature, with changing fields of view (FoVs) and user synchronization requirements, creates various challenges for edge caching. We address the lack of suitable benchmarks and propose a framework that simulates multiuser VR scenarios while logging users' interaction with objects within their actual and predicted FoVs. The benchmark's activity log can then be played back through an edge cache to assess the resulting QoEs. This tool fills a gap by supporting research in the optimization of edge caching (and other edge-cloud functions) for VR streaming.

Index Terms: Virtual Reality, Unity, Benchmark, Edge Cloud, Cache

1 INTRODUCTION

Virtual reality (VR) has advanced rapidly, driven by the demand for immersive experiences in entertainment, remote work and education [6]. However, VR's data-intensive nature requires low-latency data delivery to sustain Quality of Experience (QoE). Traditional cloud architectures struggle with these demands, especially for geographically dispersed users [7,8]. Edge computing addresses these challenges by caching content closer to users, reducing latency, and improving QoE [9]. However, the dynamic nature of VR, including changing fields of view (FoVs) and diverse user needs, complicates the design and evaluation of edge-caching strategies.

We present a preliminary VR benchmark to evaluate edge-cloud systems that support VR streaming. It features an open-source simulated VR universe with 3D user FoV motion and interactive multiuser environments. The user movement is simulated to avoid human subjects (including Institutional Review Board (IRB)) issues. The benchmark logs objects entering user FoVs (and optionally predicted FoVs), annotated with details like object-avatar distances for determining delivered resolution. These logs generate a demand process (VR workload) input for the edge cloud, with FoVs re-rendered using content from the simulated edge cloud, accounting for delivery delays and resolution degradation. This enables QoE assessment under various edge-cloud configurations.

2 METHODOLOGY

We describe the mechanics of **an example** Unity based VR simulation involving "groupie" (follower) and "principal" (leader) users

who navigate through the created 3D universe of ShapeNet objects [3]. The groupies maintain proximity to a principal so that they are part of the entourage of their nearest principal by Euclidean distance. **For example**, principal moves in straight lines at a constant velocity $\mathbf{v} \in \mathbb{R}^3$. Upon reaching the boundaries of the universe, the principal bounces off at a random angle, altering the direction while keeping the velocity $\mathbf{v}$ constant. So, if the position of the principal at time t is $\mathbf{p}_t$ and it is far from the boundary, the position at a small time increment d seconds later ($d > 0$) is simply given by $\mathbf{p}_{t+d} = \mathbf{p}_t + \mathbf{v}d$. The position of a groupie, $\mathbf{g}$, changes over a short period of time d to $\mathbf{g}_{t+d} = \mathbf{g}_t + \mathbf{v}_t d$, where $\mathbf{v}_t$ is the velocity of the groupie at time t . The velocity is updated as $\mathbf{v}_{t+d} = \mathbf{v}_t + (\mathbf{a} + \mathbf{f}) \cdot d$. Here, the attraction vector $\mathbf{a}$ pulls the groupie towards the nearest principal and is defined as $\mathbf{a} = \min(F_{\max}, \|\mathbf{p} - \mathbf{g}\|) \cdot (\mathbf{p} - \mathbf{g}) / \|\mathbf{p} - \mathbf{g}\|$, where $F_{\max}$ is the maximum force magnitude, $\mathbf{p}$ is the position of the principal and $\|\cdot\|$ is Euclidean distance. The diffusion vector $\mathbf{f} = (f_1, f_2, f_3)$ where the f_i are independent and uniformly random in $[-\delta, \delta]$.

A principal/groupie's Field of View (FoV) is defined by a minimum distance from the user at which the FoV is projected and its angular width. Each user also has a *predicted* FoV, which typically is a larger region encompassing the user's *immediate* FoV to account for potential changes in viewing direction. Even without moving, a user's FoV can shift dynamically due to rapid head movements, often referred to as "rubbernecking." These shifts are modeled by periodically adding angular perturbations to the FoV vector. The perturbations follow a normal distribution, $N(0, s^2)$. The update period for these shifts is adjustable and can be set to be much longer than the characteristic time d , which governs user motion in the VR environment. Further details on system implementation can be found in [1].

All aspects of this example, including user movement model, number of users, and number, location and size of the ShapeNet objects, are easily modified by the researcher.

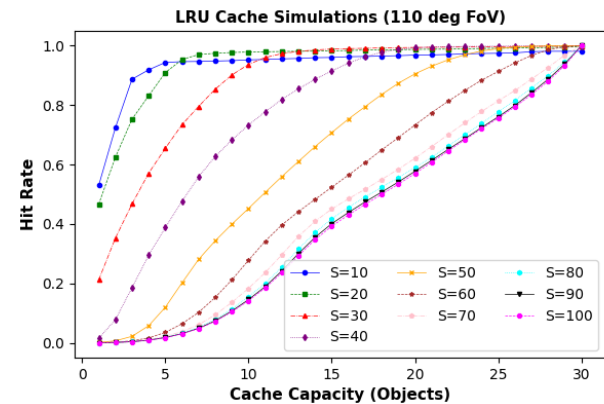


Figure 1: Hit rate of shared cache to all users at different depths (cutoffs) of the FoV at 110°.

*e-mail: nna5040@psu.edu

†e-mail: gik2@psu.edu

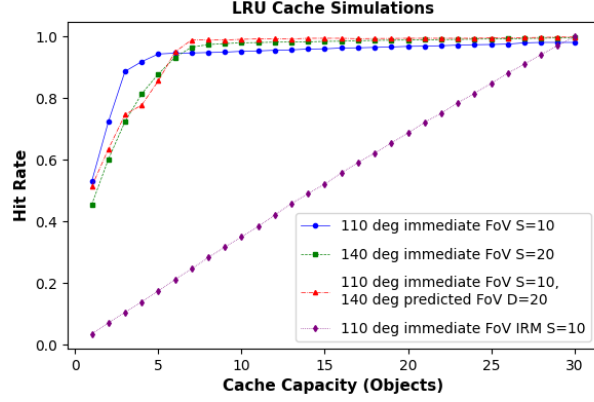


Figure 2: Hit rate of shared cache to all users at three different cache access behaviors. The blue line with circle marker only considers the immediate FoV with angle of 110 degrees and depth $S = 10$. The green line with square markers considers the immediate FoV with 140 angle degrees and depth $S = 20$. The red line with triangle markers considers both FoVs, immediate and predicted, with objects being pre-fetched at the cache when they are at the predicted FoV. Lastly, the purple line with diamond markers represent the IRM results.

3 RESULTS

We simulate a shared cache for VR users, where requests are made to fetch objects upon entering a user's FoV. A cache miss prompts the edge to retrieve the object from a remote host, update the cache, and fulfill the request. Our experiments define three VR user demand patterns based on FoV states. The FoV is categorized into the Immediate FoV (110° angle, depth S) and the Predicted FoV (140° angle, depth D), with the latter being a superset of the former. The Predicted FoV includes objects anticipated to enter the user's view, influencing request patterns and caching. These patterns are: *Standard FoV Requests* (objects entering the Immediate FoV), *Extended FoV Requests* (objects within the larger Predicted FoV), and a *Prefetching Strategy* (proactively fetching objects in the Predicted FoV to the edge cache). Figure 1 illustrates the cache hit rates for the standard FoV requests when objects are within depth (cutoff) S units from the user. As S increases, more "unnecessary" objects may be stored in the cache; leading to the degradation of hit rates.

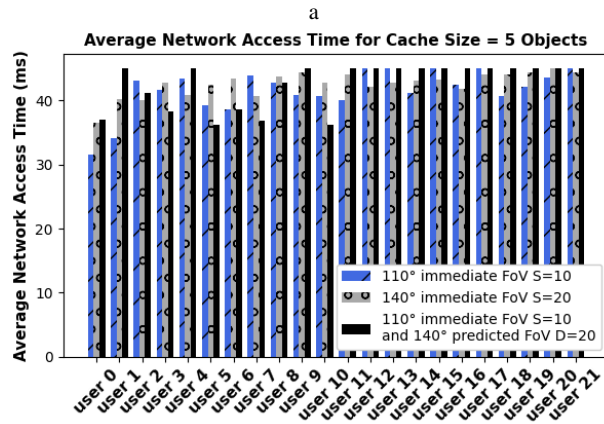


Figure 3: Average per user network access delay with a cache size of 5 objects, with three cache behaviors for each user.

Figure 2 illustrates the cache hit rates for the three different strategies. We vary the depth of the FoVs (both immediate, 110° depth S , and predicted, 140° depth D) and evaluate the shared cache hit rates. We also compare hit rates obtained from the benchmark to the cache hit rates from a commonly used theoretical Independent Reference Model (IRM) for (standard) requests that are in the immediate FoV; i.e., under the IRM, the queries for each object in the system follows an independent Poisson process conditioned on the total number of queries for each object are the same as those logged for simulated user movement. We generate delays for fetching objects from [2] for wireless access, and additionally for cache misses, from [5] for round-trips to the "closest" AWS regions. A user's QoE subject to such object-fetching delays can then be assessed. In terms of average object delays per user, Figure 3 compares the three distinct cache-access strategies.

4 CONCLUSION

We introduce a preliminary VR streaming benchmark to evaluate cache performance at the edge, particularly for multiple mobile users. By simulating user movements with dual cameras representing immediate and predicted fields of view (FoVs), the benchmark simulates the dynamic retrieval of 3D objects from the nearby edge cloud (i.e., on a cache hit) or from the remote cloud (i.e., on a cache miss). The benchmark is flexible and adaptable to third-party VR applications in Unity, allowing, e.g., for customization of user motion models to fit the applications requirements and expected user behaviors. Ongoing work is incorporating the Wave Function Collapse (WFC) algorithm [4] to dynamically generate 2D and 3D user pathways in another created virtual environment, and exploring more complex environments such as cityscapes. We will also support delivery of visible objects at different resolutions depending on their distance to the user.

GITHUB LINK TO OPEN-SOURCE SOFTWARE

The benchmark can be found in a GitHub repository at <https://github.com/PSU-Cloud/XRFlux>.

ACKNOWLEDGMENTS

This work is supported by NSF CNF grant 2212201.

REFERENCES

- [1] Nader Alfares and George Kesidis. Xrflux: Virtual reality benchmark for edge caching systems, 2024.
- [2] S. Andreev, A. Larmo, M. Gerasimenko, V. Petrov, O. Galinina, T. Tirronen, J. Torsner, and Y. Koucheryavy. Efficient small data access for machine-type communications in LTE. In *Proc. IEEE ICC*, June 2013.
- [3] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. ShapeNet: An Information-Rich 3D Model Repository, 2015.
- [4] Darui Cheng, Honglei Han, and Guangzheng Fei. Automatic generation of game levels based on controllable wave function collapse algorithm. In *Proc. Int'l Conf. on Entertainment Computing (ICEC)*, 2020.
- [5] cloudping. <https://cloudping.info>.
- [6] Yong Min Kim, Ilsun Rhiu, and Myung Hwan Yun. A systematic review of a virtual reality system from the perspective of user experience. *Int'l J. of Human-Computer Interaction*, 36(10):893–910, 2020.
- [7] Mahadev Satyanarayanan. The emergence of edge computing. *Computer*, 50(1):30–39, 2017.
- [8] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.
- [9] Shuo Wang, Xing Zhang, Yan Zhang, Lin Wang, Juwo Yang, and Wenbo Wang. A survey on mobile edge networks: Convergence of computing, caching and communications. *IEEE Access*, 5, 2017.